



دانشگاه صنعتی امیرکبیر
دانشکده مهندسی کامپیوتر



«مبانی و کاربردهای هوش مصنوعی ترم بهار ۱۴۰۵»

پروژه دوم

CSP, MDP & RL and BayesNet

در انجام پروژه‌ها به نکات زیر توجه فرمائید:

- ۱- زبان برنامه‌نویسی برای انجام این پروژه، تنها می‌تواند پایتون باشد.
- ۲- مطابق قوانین دانشگاه هر نوع کپی‌برداری و اشتراک کار دانشجویان غیرمجاز است و انجام پروژه‌ها باید به صورت انفرادی انجام شود؛ در صورت مشاهده چنین مواردی با طرفین شدیداً برخورد خواهد شد.
- ۳- توجه نمایید که بخشی از نمره اصلی شما به گزارش‌کاری که طبق موارد خواسته‌شده برای این پروژه نگارش می‌کنید اختصاص دارد
- ۴- در صورت هر گونه سوال و یا ابهام از طریق ایمیل زیر می‌توانید در ارتباط باشید:
amir.b@aut.ac.ir
- ۵- توجه نمائید فایل پاسخ پروژه‌ها تنها در صورت آپلود در سامانه کورسز پذیرفته خواهد شد و ارسال از طریق ایمیل و یا روش‌های دیگر بررسی نخواهد شد.
- ۶- فایل پروژه خود را با فرمت StudentID_P02.zip تا ساعت ۲۳:۵۹ روز ۱۴۰۵/۰۴/۲۴ فقط در بخش مربوطه در سایت درس آپلود نمائید.

"فهرست مطالب پروژه"

بخش اول مقدمه و نکات مورد نیاز	۳
۱-۱ فهرست فایل‌های شروع تمرین	۳
۲-۱ پیش‌نیاز اجرا	۴
۳-۱ قوانین کلی پروژه	۴
۴-۱ مواردی که نباید انجام دهید	۵
۵-۱ دستورکار کلی انجام پروژه	۵
۱-۵-۱ اجرای اولیه	۵
۲-۵-۱ پیدا کردن TODOها	۶
۳-۵-۱ خواندن داده‌های مسئله	۶
۴-۵-۱ کامل کردن تابع‌های کوچک‌تر	۶
۵-۵-۱ تست الگوریتم با اجرای بازی	۶
۶-۵-۱ بررسی خروجی و نوشتن گزارش	۷
۶-۱ قالب پیشنهادی گزارش نهایی	۷
بخش دوم: Chronos Relay Forge - مدل CSP	۸
۱-۲ هدف مرحله	۸
۲-۲ داستان بازی	۸
۳-۲ داده‌های مهم مسئله	۸
۴-۲ قیود اصلی	۹
۵-۲ تابع‌هایی که باید کامل شوند	۱۰
۶-۲ دستور کار دقیق CSP	۱۰
۱-۶-۲ انتخاب متغیر	۱۰
۲-۶-۲ ترتیب مقدارها	۱۱
۳-۶-۲ Backtracking Search	۱۱
۷-۲ helperهای آماده در CSP	۱۲
۸-۲ معیار موفقیت CSP	۱۲
۹-۲ پرسش‌های گزارش CSP	۱۳
بخش سوم: Chronos Gatewalk - مدل MDP	۱۴
۱-۳ هدف مرحله	۱۴
۲-۳ داستان بازی	۱۴
۳-۳ داده‌های مهم MDP	۱۴
۴-۳ تابع‌هایی که باید کامل شوند	۱۵
۵-۳ دستور کار دقیق MDP	۱۵
۱-۵-۳ مدل گذار تصادفی	۱۵
۲-۵-۳ محاسبه Q-value	۱۶

۱۶.....	۳-۵-۳ یک sweep از Value Iteration
۱۶.....	۴-۵-۳ Policy Iteration
۱۷.....	۶-۳ helperهای آماده در MDP
۱۷.....	۷-۳ کنترل دستی
۱۸.....	۸-۳ معیار موفقیت MDP
۱۸.....	۹-۳ پرسش‌های گزارش MDP
۱۹.....	بخش چهارم: Chronos Shard Trial - یادگیری تقویتی
۱۹.....	۱-۴ هدف مرحله
۱۹.....	۲-۴ داستان بازی
۱۹.....	۳-۴ داده‌های مهم RL
۲۰.....	۴-۴ تابع‌هایی که باید کامل شوند
۲۰.....	۵-۴ دستور کار RL
۲۰.....	۱-۵-۴ epsilon-greedy انتخاب کنش
۲۰.....	۲-۵-۴ اجرای یک episode
۲۱.....	۳-۵-۴ Q-Learning
۲۱.....	۴-۵-۴ SARSA
۲۱.....	۵-۵-۴ گزارش episode
۲۱.....	۶-۴ helperهای آماده در RL
۲۲.....	۷-۴ معیار موفقیت RL
۲۳.....	۸-۴ پرسش‌های گزارش RL
۲۴.....	بخش پنجم: Chronos Signal Hunt - شبکه بیزی
۲۴.....	۱-۵ هدف مرحله
۲۴.....	۲-۵ داستان بازی
۲۴.....	۳-۵ داده‌های مهم Bayes Net
۲۵.....	۴-۵ تابع‌هایی که باید کامل شوند
۲۵.....	۵-۵ دستور کار دقیق Bayes Net
۲۵.....	۱-۵-۵ محاسبه likelihood برای Flux
۲۵.....	۲-۵-۵ محاسبه likelihood برای Distance
۲۶.....	۳-۵-۵ Exact Inference
۲۶.....	۴-۵-۵ استفاده از posterior
۲۷.....	۶-۵ معیار موفقیت Bayes Net
۲۷.....	۷-۵ پرسش‌های گزارش Bayes Net
۲۸.....	گزارش پروژه
۲۹.....	تست‌های پیشنهادی

در این پروژه باید چهار مرحله از پروژه Chronos را کامل کنید. هر مرحله یک بازی کوچک است که یک مفهوم اصلی هوش مصنوعی را به شکل قابل مشاهده نشان می‌دهد. هدف فقط گرفتن جواب نهایی نیست؛ باید مدل را درست تعریف کنید، الگوریتم را پیاده‌سازی کنید، خروجی بازی را تحلیل کنید و بتوانید توضیح دهید چرا الگوریتم شما به آن نتیجه رسیده است.

۱-۱ فهرست فایل‌های شروع تمرین

فایل	نقش
<code>chronos_csp_todo.py</code>	مرحله اول، حل مسئله قیدی در بازی Chronos Relay Forge
<code>chronos_mdp_todo.py</code>	مرحله دوم، تصمیم‌گیری مارکوفی در بازی Chronos Gatewalk
<code>chronos_rl_todo.py</code>	مرحله سوم، یادگیری تقویتی در بازی Chronos Shard Trial
<code>chronos_bayesnet_todo.py</code>	مرحله چهارم، استنتاج بیزی در بازی Chronos Signal Hunt
<code>Source.png</code>	تصویر منبع برای tileها، آیتم‌ها و کاراکترهای بازی
<code>requirements.txt</code>	وابستگی‌های لازم برای اجرای پروژه

۲-۱ پیش‌نیاز اجرا

ابتدا وابستگی‌ها را نصب کنید:

```
pip install -r requirements.txt
```

سپس هر مرحله را جداگانه اجرا کنید اگر بازی باز شد ولی دکمه حل یا آموزش پیام **TODO** نشان داد، یعنی فایل درست اجرا شده و شما باید تابع‌های خالی را کامل کنید.

۳-۱ قوانین کلی پروژه

۱. فایل‌های **todo** را کامل کنید؛ تحویل نهایی باید همین فایل‌ها باشند.
۲. از **hard-code** کردن جواب نهایی، مسیر نهایی، **assignment** نهایی یا **posterior** نهایی خودداری کنید.
۳. رابط کاربری، ابعاد پنجره، نقشه، **Source.png** و داده‌های اصلی بازی را بدون دلیل آموزشی تغییر ندهید.
۴. تغییرات شما باید داخل تابع‌های الگوریتمی باشد، مگر اینکه برای جلوگیری از خطای منطقی یا گزارش بهتر مجبور باشید اصلاح کوچکی انجام دهید.
۵. بعد از تکمیل هر بخش، بازی باید بدون **crash** اجرا شود.
۶. خروجی هر الگوریتم باید قابل تکرار و قابل توضیح باشد.
۷. اگر یک **helper function** در کد وجود دارد، قبل از نوشتن کد جدید بررسی کنید که همان **helper** نیاز شما را برطرف می‌کند یا نه.
۸. برای هر بخش گزارش لازم است و فقط کد و نتایج کافی نیست.

۴-۱ مواردی که نباید انجام دهید

- جواب را مستقیم داخل دکمه‌ها قرار ندهید.
- از روی مختصات ثابت، مسیر یا جواب را تقلبی نسازید.
- **Source.png** را تغییر ندهید.
- برای حل مسئله از sleep، delay یا random output به جای الگوریتم استفاده نکنید.
- در CSP، فقط اولین assignment ظاهراً درست را بدون بررسی quota و cost قبول نکنید.
- در MDP، policy را دستی ننویسید؛ باید از value یا policy iteration به دست بیاید.
- در RL، مسیر را دستی ذخیره نکنید؛ باید از Q-value ها و تجربه محیط یاد گرفته شود.
- در Bayes Net، مکان پنهان را از **hidden** برای inference استفاده نکنید. **hidden**
- فقط برای تولید شواهد و بررسی نتیجه بازی است.

۵-۱ دستور کار کلی انجام پروژه

۱-۵-۱ اجرای اولیه

هر چهار فایل TODO را اجرا کنید. هدف این مرحله این است که مطمئن شوید محیط، pygame و assetها درست کار می‌کنند.

انتظار می‌رود:

- صفحه بازی باز شود.
- دکمه‌ها دیده شوند.
- بعضی دکمه‌ها فعلاً پیام **TODO** بدهند.
- بازی crash نکند.

۱-۵-۲ پیدا کردن TODOها

در هر فایل، عبارت‌های زیر را جست‌وجو کنید:

TODO

NotImplementedError

هر جایی که **NotImplementedError** دارد، همان نقطه اصلی تمرین است. لازم نیست کل بازی را از اول بنویسید.

۱-۵-۳ خواندن داده‌های مسئله

قبل از کدنویسی، داده‌های ثابت همان فایل را بخوانید:

- در CSP، فرکانس‌ها، سهمیه‌ها، هزینه‌ها، رله‌ها و اتصال‌ها.
- در MDP، دیوارها، terminal، gate، charger، noise و rewardها.
- در RL، trap، shard، slip، goal و Q-table.
- در Bayes Net، beacon، rift zone، prior و observationها.

۱-۵-۴ کامل کردن تابع‌های کوچک‌تر

اول تابع‌های کوچک‌تر را کامل کنید. برای مثال:

- در CSP، اول **select_unassigned** و **ordered_values**.
- در MDP، اول **transitions** و **q_value**.
- در RL، اول **choose_action**.
- در Bayes Net، اول **likelihood**.

بعد سراغ تابع اصلی بروید.

۱-۵-۵ تست الگوریتم با اجرای بازی

بعد از تکمیل هر تابع، بازی همان مرحله را اجرا کنید و دکمه مربوط را بزنید. لازم نیست صبر کنید کل پروژه تمام شود.

۱-۵-۶ بررسی خروجی و نوشتن گزارش

برای هر بخش، حداقل یک خروجی موفق ثبت کنید. گزارش شما باید توضیح دهد:

- state یا variableها چه هستند.
- action یا valueها چه معنی دارند.
- الگوریتم چگونه تصمیم می‌گیرد.
- نتیجه نهایی چه بود.
- اگر چند روش وجود دارد، کدام بهتر عمل کرد و چرا.

۱-۶ قالب پیشنهادی گزارش نهایی

برای هر چهار بخش می‌توانید یک گزارش با ساختار زیر بنویسید:

نام بخش:

مدل مسئله:

تابع‌های تکمیل‌شده:

الگوریتم استفاده‌شده:

خروجی مشاهده‌شده:

تجزیه و تحلیل آن:

یک مشکل یا نکته جالب:

پاسخ به سوالات انتهایی هر بخش:

و در نهایت پاسخ به سوالات مقایسه‌ای انتهایی فایل و سایر مواردی که از نظرتان برای گزارش پروژه ضروری و مورد نیاز است.

بخش دوم: Chronos Relay Forge – مدل CSP

۱-۲ هدف مرحله

در این مرحله باید یک مسئله ارضای محدودیت را حل کنید. ده رله زمانی در نقشه وجود دارد. هر رله باید یکی از چهار فرکانس زیر را بگیرد:

- Cyan -
- Green -
- Violet -
- Gold -

اما انتخاب فرکانسها آزاد نیست. هر رله domain مخصوص خود را دارد و بین بعضی رلهها constraint وجود دارد.

۲-۲ داستان بازی

Chronos باید ده رله زمانی را روشن کند. هر رله با یک rune یا فرکانس فعال می شود. اگر دو رله متصل، rune ناسازگار داشته باشند، شبکه زمانی ناپایدار می شود. علاوه بر این، دفتر انرژی یا ledger فقط تعداد مشخصی از هر فرکانس را قبول می کند.

۳-۲ داده های مهم مسئله

در فایل `chronos_csp_todo.py` این داده ها را می بینید:

```
FREQ_COSTS = {"Cyan": 3, "Green": 2, "Violet": 4, "Gold": 5}
```

```
FREQ_QUOTAS = {"Cyan": 3, "Green": 2, "Violet": 2, "Gold": 3}
```

نام داستانی رلهها:

نام	کد
Astra	A
Boreal	B
Cyra	C
Dusk	D
Echo	E
Fable	F
Gale	G
Halo	H
Ion	I
Juno	J

۲-۴ قیود اصلی

۱. هر رله دقیقاً یک فرکانس می‌گیرد.
 ۲. رله‌های متصل نباید فرکانس یکسان داشته باشند.
 ۳. تعداد استفاده از هر فرکانس باید دقیقاً برابر quota آن باشد.
 ۴. بعضی رله‌ها فرکانس ممنوع دارند:
- Boreal فرکانس Violet را نمی‌پذیرد.
 - Dusk فرکانس Gold را نمی‌پذیرد.
 - Fable فرکانس Cyan را نمی‌پذیرد.

- Halo فرکانس Green را نمی‌پذیرد.

- Ion فرکانس Cyan را نمی‌پذیرد.

- Juno فرکانس Violet را نمی‌پذیرد.

۵. قانون داستانی: Cyra-Halo

- این جفت نباید هم‌زمان روی ترکیب ناسازگار Violet/Gold قرار بگیرند.

۶. قانون داستانی: Dusk-Juno

- این جفت نباید هم‌زمان روی ترکیب ناسازگار Cyan/Gold قرار بگیرند.

۵-۲ تابع‌هایی که باید کامل شوند

در فایل `chronos_csp_todo.py` این تابع‌ها TODO هستند:

```
select_unassigned(self, assignment, domains, mode)
```

```
ordered_values(self, var, assignment, domains, use_lcv)
```

```
backtracking_search(self, algorithm, variable_mode="plain", use_lcv=False)
```

۶-۲ دستور کار دقیق CSP

۱-۶-۲ انتخاب متغیر

تابع `select_unassigned` باید متغیر بعدی را انتخاب کند.

حالت‌های لازم:

- اگر `mode == "plain"` یا حالت پیش‌فرض بود، اولین رله `assign` نشده را برگردانید.

- اگر `mode == "mrv"` بود، رله‌ای را انتخاب کنید که کمترین مقدار ممکن در `domain`

فعلی دارد.

- اگر `mode == "degree"` بود، اول `MRV` را اعمال کنید و اگر چند رله برابر بودند،

رله‌ای را انتخاب کنید که بیشترین همسایه `assign` نشده دارد.

نکته: $MRV = \text{Minimum Remaining Values}$

$\text{Degree} = \text{انتخاب متغیری که بیشترین اثر را روی بقیه متغیرها دارد}$

۲-۶-۲ ترتیب مقدارها

تابع `ordered_values` باید مقدارهای `domain` یک رله را مرتب کند.

حالت‌های لازم:

- اگر `use_lcv == False` بود، مقدارها را با ترتیب ساده و قابل دفاع برگردانید.
- اگر `use_lcv == True` بود، مقدارهایی را جلوتر بیاورید که کمترین مقدار را از `domain` همسایه‌ها حذف می‌کنند.

نکته: $LCV = \text{Least Constraining Value}$

برای محاسبه `LCV` می‌توانید برای هر رنگ بشمارید چند مقدار از `domain` همسایه‌ها با انتخاب آن رنگ غیرمجاز می‌شوند.

۳-۶-۲ Backtracking Search

تابع `backtracking_search` باید یک جست‌وجوی بازگشتی کامل انجام دهد.

در هر مرحله:

۱. اگر `assignment` کامل شد، `quota` را بررسی کنید.
۲. اگر هزینه `assignment` از بهترین جواب فعلی بیشتر یا برابر شد، آن شاخه را `prune` کنید.
۳. اگر با مقدارهای باقی‌مانده دیگر امکان رسیدن به `quota` وجود ندارد، آن شاخه را `prune` کنید.
۴. متغیر بعدی را با `select_unassigned` انتخاب کنید.
۵. مقدارها را با `ordered_values` مرتب کنید.
۶. مقدار را با `constraint` های `unary` و `binary` بررسی کنید.
۷. پس از `forward_check`، `assignment` را اجرا کنید.

۸. اگر domain یک همسایه خالی شد، backtrack کنید.

۹. آمار **backtracks**، **nodes** و **checks** را ثبت کنید.

۷-۲ helperهای آماده در CSP

قبل از نوشتن کد جدید، این تابع‌ها را بررسی کنید:

تابع	کاربرد
quota_counts	شمارش تعداد استفاده از هر فرکانس
quota_possible	بررسی اینکه آیا هنوز رسیدن به quota ممکن است
unary_ok	بررسی ممنوعیت فرکانس برای یک رله
constraint_ok	بررسی constraint بین دو رله
consistent	بررسی سازگاری مقدار جدید با assignment فعلی
forward_check	حذف مقدارهای ناسازگار از domain همسایه‌ها
complete	بررسی کامل بودن quota و assignment
total_cost	محاسبه هزینه کل فرکانس‌ها

۸-۲ معیار موفقیت CSP

پس از تکمیل، باید بتوانید دکمه‌های زیر را تست کنید:

Solve -

LCV -

- MRV - Degree

انتظار می‌رود:

- هر ده رله مقدار بگیرند.
- conflict صفر باشد.
- ledger یا quota دقیقاً کامل شود.
- گزارش تعداد node و backtrack نمایش داده شود.

۹-۲ پرسش‌های گزارش CSP

- طبق پیاده‌سازی و خروجی‌هایی که برای این بخش داشتید؛ در گزارش خود پاسخ دهید:
- آیا کمترین هزینه همیشه با اولین جواب پیدا می‌شود؟ چرا؟
 - اگر quota حذف شوند، مسئله ساده‌تر می‌شود یا سخت‌تر؟ توضیح دهید.
 - چرا وجود quota باعث می‌شود مسئله از یک رنگ‌آمیزی ساده graph سخت‌تر شود؟
 - Degree heuristic وقتی چند متغیر domain هم‌اندازه دارند، چه کمکی می‌کند؟
 - forward checking چه زمانی باعث prune شدن یک شاخه می‌شود؟
 - اگر فقط conflict‌های محلی را بررسی کنیم ولی quota را چک نکنیم، چه نوع جواب اشتباهی ممکن است تولید شود؟
 - با توجه به داده‌های مسئله، فرکانس Gold بیشترین هزینه و Green کمترین هزینه را دارد. در اجرای الگوریتم شما، مکانیزم Pruning بر اساس هزینه کل چگونه توانست شاخه‌هایی که فرکانس‌های گران‌قیمت دارند را سریع‌تر هرس کند؟ آیا این هرس روی تعداد کل نودهای بررسی‌شده تأثیر ملموسی داشت؟
 - با توجه به خروجی پیاده‌سازی شما، فعال کردن هیوریستیک LCV چه تغییر عددی در تعداد backtracks ایجاد کرد؟ با توجه به زمان اجرای الگوریتم، آیا سربار محاسباتی محاسبه‌ی LCV در این نقشه به صرفه بود؟

بخش سوم: Chronos Gatewalk – مدل MDP

۱-۳ هدف مرحله

در این مرحله باید یک Markov Decision Process را حل کنید. Chronos روی یک grid حرکت می‌کند، اما حرکت قطعی نیست. اگر فرمان شرق بدهید، ممکن است واقعاً شرق برود یا به دلیل drift به یکی از جهتهای کناری بلغزد.

۲-۳ داستان بازی

Chronos باید در یک مسیر خطرناک به مقصد برسد. بعضی خانه‌ها charger هستند و Chronos شارژ می‌کند. بعضی gateها فقط وقتی باز می‌شوند که Chronos قبلاً شارژ شده باشد. terminalها پایان مسیر هستند و پاداش مثبت یا منفی دارند.

۳-۳ داده‌های مهم MDP

در فایل `chronos_mdp_todo.py` داده‌های زیر وجود دارند:

```
walls = {(1, 1), (3, 1), (3, 2), (5, 3), (1, 5), (4, 5)}
terminals = {(7, 0): 12.0, (7, 6): -12.0, (0, 6): 4.0}
hazards = {(2, 4): -2.0, (6, 4): -3.0, (5, 1): -1.5, (4, 3): -2.5}
chargers = {(2, 6), (4, 0)}
gates = {(6, 2), (6, 5)}
start = (0, 0, False)
```

حالت‌ها به صورت روبه‌رو هستند: State = (x, y, charged)

کنش‌ها چهار جهت اصلی‌اند: N, E, S, W

۳-۴ تابع‌هایی که باید کامل شوند

در فایل `chronos_mdp_todo.py` این تابع‌ها TODO هستند:

```
transitions(self, state, action)
q_value(self, state, action, values=None)
value_iteration_step(self)
policy_iteration(self, limit=20)
```

۳-۵ دستور کار دقیق MDP

۳-۵-۱ مدل گذار تصادفی

تابع `transitions` باید همه حالت‌های بعدی ممکن را برای یک `state` و `action` برگرداند. خروجی تابع باید لیستی از سه‌تایی‌ها باشد:

(next_state, probability, reward)

قانون حرکت:

- اگر حالت terminal است، به خودش برمی‌گردد.
- حرکت اصلی با احتمال `noise - 1` رخ می‌دهد.
- لغزش به جهت کناری چپ با احتمال `noise / 2` رخ می‌دهد.
- لغزش به جهت کناری راست با احتمال `noise / 2` رخ می‌دهد.
- اگر چند حرکت به یک `next_state` برسند، احتمال‌هایشان باید merge شوند.

نکته تست:

جمع احتمال‌های `transitions` برای هر `state/action` باید تقریباً ۱ باشد.

۲-۵-۳ محاسبه Q-value

تابع **q_value** باید Bellman backup را برای یک state-action حساب کند:

$$Q(s, a) = \sum P(s' | s, a) * [R(s, a, s') + \gamma * V(s')]$$

اگر **values** داده نشده بود، از **self.values** استفاده کنید. اگر داده شده بود، همان dictionary را مبنا قرار دهید.

۳-۵-۳ یک sweep از Value Iteration

تابع **value_iteration_step** باید روی همه stateها حرکت کند و مقدار جدید هر state را حساب کند:

$$V_new(s) = \max_a Q(s, a)$$

برای terminalها مقدار را صفر نگه دارید، چون reward ورود به terminal در transition قبلی داده می‌شود.

در پایان:

- **self.values** را به‌روز کنید.
- **policy** را با بهترین action هر state به‌روز کنید.
- **self.iterations** را افزایش دهید.
- **self.last_delta** را برابر بیشترین تغییر value قرار دهید.
- یک **MdpReport** مناسب برگردانید.

۴-۵-۳ Policy Iteration

تابع **policy_iteration** باید دو کار را تکرار کند:

۱. **policy evaluation**: مقدار policy فعلی را با چند sweep تقریبی حساب کنید.
۲. **policy improvement**: برای هر state بهترین action جدید را پیدا کنید.

اگر هیچ action تغییر نکرد، policy stable شده است.

۳-۶ helperهای آماده در MDP

تابع	کاربرد
states	تولید همه stateهای ممکن
pos	گرفتن مختصات از state
terminal	بررسی terminal بودن state
reward	محاسبه reward ورود به خانه بعدی
move	حرکت deterministic با رعایت دیوار و gate
side_actions	جهت‌های لغزش چپ و راست
best_action	انتخاب action با بیشترین Q-value
update_policy	به‌روزرسانی policy از روی valueها
evaluate_policy	چند sweep برای evaluation

۳-۷ کنترل دستی

پس از کامل شدن **transitions**، می‌توانید با کلیدهای زیر Chronos را دستی حرکت دهید:

- Arrow keys

- W, A, S, D

هدف از حرکت دستی این است که ببینید policy پیشنهادی واقعاً در محیط تصادفی چه رفتاری دارد.

۸-۳ معیار موفقیت MDP

پس از تکمیل:

- دکمه **Step** باید یک sweep انجام دهد.
- دکمه **Solve** باید value iteration را تا همگرایی یا limit اجرا کند.
- دکمه **Route** باید policy iteration اجرا کند.
- فلش‌های روی grid باید مسیر معقول نشان دهند.
- تغییر **Drift** باید روی policy اثر بگذارد.
- اگر Chronos به terminal برسد، بازی باید متوقف شود و پیام پایان نشان دهد.

۹-۳ پرسش‌های گزارش MDP

طبق پیاده‌سازی و خروجی‌هایی که برای این بخش داشتید؛ در گزارش خود پاسخ دهید:

- state در این مسئله چرا فقط مختصات نیست و charged هم لازم دارد؟
- چرا terminalها مقدار value صفر دارند ولی reward مثبت یا منفی همچنان دیده می‌شود؟ تفاوت reward ورود به terminal با value خود terminal چیست؟
- noise یا drift چگونه می‌تواند مسیر بهینه را تغییر دهد؟
- value iteration و policy iteration در این نقشه چه تفاوتی در تئوری، سرعت همگرایی و سیاست نهایی در خروجی‌هایتان داشتند؟
- آیا مسیر کوتاه‌تر همیشه بهتر است؟ با توجه به hazard و gate توضیح دهید.
- چرا باید outcomeهای تکراری در transition merge شوند؟
- اگر مقدار پارامتر noise را در کد خود تغییر دهید، سیاست نهایی عامل در خانه‌های مجاور خطر (hazard) چه تغییری می‌کند؟ رفتار عامل را در حالت محیط کاملاً قطعی ($\text{Noise} = 0$) با حالت محیط تصادفی تحلیل کنید.

بخش چهارم: Chronos Shard Trial – یادگیری تقویتی

۱-۴ هدف مرحله

در این مرحله عامل از قبل transition model کامل ندارد. باید با تجربه کردن محیط، مقدار action را یاد بگیرد. شما باید Q-Learning و SARSA را در یک محیط grid پیاده‌سازی کنید.

۲-۴ داستان بازی

Chronos باید سه shard را جمع کند و بعد به core نهایی برسد. اگر بدون shardهای کامل به goal برسد، جریمه می‌گیرد. trapها خطرناک هستند و بعضی خانه‌ها slow zone هستند. حرکت نیز slip دارد، یعنی action انتخاب‌شده همیشه دقیقاً اجرا نمی‌شود.

۳-۴ داده‌های مهم RL

در فایل `chronos_rl_todo.py` داده‌های زیر وجود دارند:

```
walls = {(1, 1), (3, 1), (6, 1), (3, 2), (6, 3), (1, 5), (4, 5), (6, 6)}
traps = {(2, 5), (5, 2), (7, 4)}
shards = [(2, 2), (4, 6), (7, 2)]
slow_zones = {(3, 4), (5, 5)}
start = (0, 7)
goal = (8, 0)
slip = 0.20
```

حالت‌ها به شکل روبه‌رو هستند: $RState = (x, y, shard_mask)$

`shard_mask` مشخص می‌کند کدام shardها جمع شده‌اند. اگر سه shard وجود داشته باشد، mask نهایی برابر 111 باینری است.

۴-۴ تابع‌هایی که باید کامل شوند

در فایل `chronos_rl_todo.py` این تابع‌ها TODO هستند:

`choose_action(self, state)`

`train_episode(self, max_steps=160)`

۵-۴ دستور کار RL

۴-۵-۱ انتخاب کنش `epsilon-greedy`

تابع `choose_action` باید یکی از دو حالت زیر را انجام دهد:

- با احتمال `epsilon` یک `action` تصادفی انتخاب کند.
 - با احتمال `1 - epsilon` بهترین `action` فعلی را از روی `Q-table` انتخاب کند.
- تابع آماده `best_action` می‌تواند بهترین `action` را برگرداند.

۴-۵-۲ اجرای یک `episode`

تابع `train_episode` باید از `start` شروع کند و تا `terminal` یا رسیدن به `max_steps` ادامه دهد.

در هر `step`:

۱. `action` را انتخاب کنید.
۲. `action` را در محیط اجرا کنید.
۳. `next_state, reward, done` را بگیرید.
۴. `target` را بر اساس الگوریتم فعلی بسازید.
۵. `Q-value` را آپدیت کنید.
۶. `state` و `action` را جلو ببرید.
۷. اگر `done` شد، `episode` تمام شود.

۳-۵-۴ Q-Learning

اگر الگوریتم Q-Learning بود:

$$\text{target} = \text{reward} + \gamma * \max_a Q(\text{next_state}, a)$$

اگر حالت terminal بود:

$$\text{target} = \text{reward}$$

سپس:

$$Q(s,a) = Q(s,a) + \alpha * (\text{target} - Q(s,a))$$

۴-۵-۴ SARSA

اگر الگوریتم SARSA بود:

ابتدا action بعدی را با همان policy انتخاب کنید:

$$\text{next_action} = \text{choose_action}(\text{next_state})$$

سپس:

$$\text{target} = \text{reward} + \gamma * Q(\text{next_state}, \text{next_action})$$

اگر حالت terminal بود:

$$\text{target} = \text{reward}$$

۵-۵-۴ گزارش episode

در پایان episode:

- `self.episodes` را افزایش دهید.
- `epsilon` را کمی کاهش دهید.
- مسیر طی شده را در `self.last_path` ذخیره کنید.
- یک `EpisodeReport` برگردانید.

۶-۴ helperهای آماده در RL

تابع	کاربرد
<code>env.reset</code>	شروع episode از start
<code>env.step</code>	اجرای action در محیط فعلی
<code>env.sample_transition</code>	حرکت تصادفی برای حالت دستی
<code>env.reward</code>	reward خانه بعدی
<code>env.terminal</code>	بررسی پایان episode
<code>value</code>	بیشترین Q-value یک state
<code>best_action</code>	action با بیشترین Q-value
<code>train_many</code>	اجرای چند episode پشت سر هم

۷-۴ معیار موفقیت RL

پس از تکمیل:

- دکمه **Run** باید یک episode آموزش اجرا کند.
- دکمه **Train50** باید ۵۰ episode پشت سر هم اجرا کند.
- مقدار **Episodes** افزایش پیدا کند.
- **epsilon** به مرور کمتر شود.

- مسیر آخر روی grid دیده شود.
- پس از آموزش کافی، agent باید تمایل بیشتری به جمع کردن shardها و رسیدن به core داشته باشد.

۸-۴ پرسش‌های گزارش RL

- طبق پیاده‌سازی و خروجی‌هایی که برای این بخش داشتید؛ در گزارش خود پاسخ دهید:
- تفاوت Q-Learning و SARSA در target چیست؟ کدام روش در این نقشه محتاط‌تر عمل کرد؟ Q-Learning یا SARSA؟ تحلیل کنید.
 - اگر slip بیشتر شود، انتظار دارید یادگیری سخت‌تر شود یا آسان‌تر؟ چرا؟
 - چرا در Shard Trial مدل کامل transition را مستقیماً حل نمی‌کنیم و به یادگیری تقویتی می‌رویم؟
 - epsilon-greedy چه تعادلی بین exploration و exploitation ایجاد می‌کند؟ چرا epsilon باید در ابتدا بزرگ‌تر باشد و بعد کاهش پیدا کند؟
 - shard mask چه نقشی در تعریف state دارد؟ اگر shard mask را از state حذف کنیم، عامل چه اشتباهی می‌کند؟
 - پس از چند بار Train50 چه تغییری در مسیر یا return مشاهده کردید؟
 - پس از آموزش کامل عامل با هر دو الگوریتم Q-Learning و SARSA، جدول Q-Value ها یا مسیر نهایی حرکت عامل را در مجاورت خانه‌های خطر (traps) مقایسه کنید. آیا مسیر بهینه پیدا شده توسط این دو الگوریتم روی نقشه کاملاً یکسان است یا یکی از آن‌ها مسیر امن‌تر (و دورتر از تله‌ها) را انتخاب کرده است؟ این تفاوت را بر اساس رفتار On-policy و Off-policy در مواجهه با پارامتر slip تحلیل کنید

بخش پنجم: Chronos Signal Hunt – شبکه بیزی

۱-۵ هدف مرحله

در این مرحله باید با شواهد ناقص و نویزی، مکان یک anomaly پنهان را حدس بزنید. مسئله به جای جست‌وجوی قطعی، نیاز به محاسبه احتمال posterior دارد.

۲-۵ داستان بازی

چهار beacon در گوشه‌های نقشه قرار دارند. هر beacon می‌تواند یک Ping یا Flux بدهد. Ping فاصله anomaly را با نویز گزارش می‌کند. Flux نشان می‌دهد آیا حوالی beacon احتمالاً انرژی فازی بالاست یا نه. anomaly هم یک موقعیت پنهان دارد و هم یک phase پنهان.

۳-۵ داده‌های مهم Bayes Net

در فایل `chronos_bayesnet_todo.py` داده‌های زیر وجود دارند:

`walls = {(1, 1), (4, 1), (6, 1), (3, 2), (1, 4), (6, 4), (3, 6), (7, 6)}`

`beacons = [(0, 0), (8, 0), (0, 7), (8, 7)]`

`rift_zones = {(2, 2), (5, 5), (7, 3)}`

حالت پنهان:

`BState = (x, y, phased)`

مشاهده:

`Observation(beacon, kind, reading)`

`kind` می‌تواند یکی از این دو باشد:

`distance` -

`flux` -

۴-۵ تابع‌هایی که باید کامل شوند

در فایل `chronos_bayesnet_todo.py` این تابع‌ها TODO هستند:

`likelihood(self, obs, state)`

`infer(self, observations)`

۵-۵ دستور کار دقیق Bayes Net

۱-۵-۵ محاسبه `likelihood` برای Flux

اگر `obs.kind == "flux"` باشد:

۱. بررسی کنید `state` به `rift zone` نزدیک است یا نه.
۲. اگر `state` فازدار باشد یا نزدیک `rift` باشد، احتمال `flux` بالا زیاد است.
۳. اگر `reading` برابر ۱ بود، احتمال `high` را برگردانید.
۴. اگر `reading` برابر ۰ بود، مکمل آن را برگردانید.

مدل پیشنهادی:

`high_probability = ۰.۷۸` اگر `phased` یا `near_rift` باشد

`high_probability = ۰.۱۸` در غیر این صورت

۲-۵-۵ محاسبه `likelihood` برای Distance

اگر `obs.kind == "distance"` باشد:

۱. فاصله `Manhattan` بین `beacon` و موقعیت `state` را حساب کنید.
۲. اگر `state` فازدار است، یک واحد به فاصله واقعی اضافه کنید.
۳. اختلاف `reading` و فاصله واقعی را حساب کنید.
۴. بر اساس اندازه خطا احتمال برگردانید.

مدل پیشنهادی:

خطا	احتمال
۰	0.62
۱	0.22
۲	0.10
۳	0.04
بیشتر	0.02

Exact Inference ۳-۵-۵

تابع **infer** باید posterior را از روی همه observationها بسازد.

برای هر state:

$$\text{unnormalized}[\text{state}] = \text{prior}[\text{state}] * \prod \text{likelihood}(\text{observation} | \text{state})$$

بعد مجموع همه احتمالات را حساب کنید و normalize کنید:

$$\text{posterior}[\text{state}] = \text{unnormalized}[\text{state}] / \text{total}$$

اگر total صفر شد، برای جلوگیری از خطا می‌توانید prior را برگردانید.

۴-۵-۵ استفاده از posterior

کدهای آماده از posterior برای موارد زیر استفاده می‌کنند:

- **position_posterior**
- **map_state**
- **map_phase**
- **entropy**
- رنگ‌آمیزی grid
- نمایش احتمال هر خانه

بنابراین اگر **posterior** درست normalize نشود، کل نمایش احتمالاتی بازی اشتباه می‌شود.

۵-۶ معیار موفقیت Bayes Net

پس از تکمیل:

- دکمه **Ping** باید یک مشاهده distance اضافه کند.
- دکمه **Flux** باید یک مشاهده flux اضافه کند.
- دکمه **Sweep** باید چند مشاهده پشت سر هم اضافه کند.
- احتمال‌ها روی grid تغییر کنند.
- MAP cell پس از چند مشاهده معنادارتر شود.
- entropy معمولاً پس از مشاهده‌ها کاهش پیدا کند.
- با کلیک روی خانه، بازی باید حدس شما را بررسی کند و پایان مناسب نشان دهد.

۵-۷ پرسش‌های گزارش Bayes Net

طبق پیاده‌سازی و خروجی‌هایی که برای این بخش داشتید؛ در گزارش خود پاسخ دهید:

- چرا posterior باید بعد از ضرب likelihoodها normalize شود؟
- Flux چه اطلاعاتی می‌دهد که Ping به تنهایی نمی‌دهد؟ چرا phase در state پنهان آمده است؟
- مفهوم entropy چه چیزی را درباره uncertainty نشان می‌دهد؟ در اجرای پروژه، با افزایش تعداد مشاهدات، نمودار یا مقدار عددی آنروپی چه تغییری کرد و این روند چه معنایی دارد؟
- آیا همیشه MAP cell همان مکان واقعی است؟ چرا ممکن است نباشد؟ اگر چند مشاهده متناقض یا نویزی داشته باشیم، posterior چگونه با آن‌ها برخورد می‌کند؟
- prior در این بازی چگونه bias شده است؟
- likelihood برای distance بر اساس چه چیزی محاسبه می‌شود؟
- در خروجی‌های ثبت‌شده‌ی شما، آیا زمانی اتفاق افتاد که به دلیل نویز شدید سنسورها، توزیع posterior روی یک سلول اشتباه همگرا شود (مقدار آنروپی پایین بیاید ولی مکان مکان واقعی نباشد)؟ در چنین سناریویی، عامل پس از دریافت چه نوع مشاهدات جدیدی متوجه اشتباه خود می‌شود و توزیع احتمال چگونه دوباره تغییر ماهیت می‌دهد؟ تحلیل کنید.

- با توجه به ساختار فرمول likelihood برای دو معیار Distance و Flux، سنسورهای بازی به طور متوسط پس از چند Step مشاهده توانستند آنتروپی سیستم را به نزدیکی صفر برسانند و مکان دقیق آنومالی را قفل کنند؟ اگر به صورت فرضی سنسور Flux کاملاً قطع می‌شد و فقط بر اساس داده‌های Distance استنتاج می‌کردید، سرعت همگرایی (تعداد گام‌های مورد نیاز) چه تغییری می‌کرد؟

گزارش پروژه

بعد از پیاده‌سازی، شما موظف به ارائه‌ی یک گزارش جامع هستید که علاوه بر تشریح دقیق ساختار کدها و روند پیاده‌سازی که انجام دادید، باید به تجزیه و تحلیل خروجی‌ها پرداخته و طبق ساختار پیشنهادی برای گزارش که در صفحه ۷ به آن اشاره شد پیش بروید و در پایان هر بخش نیز، سوالات مربوطه را با تجزیه و تحلیل با توجه به پیاده‌سازی و نتایجی که بدست آوردید پاسخ دهید.

در نهایت و در انتهای فایل گزارش خود نیز برای درک بهتر روند پیاده‌سازی و نتایجی که بدست آورده‌اید؛ پاسخ سوالات مقایسه‌ای زیر را نیز با تجزیه و تحلیل بنویسید:

- CSP و MDP هر دو با state یا assignment سروکار دارند، اما تفاوت هدف آن‌ها چیست؟
- MDP و RL هر دو policy تولید می‌کنند. تفاوت اصلی روش تولید policy در این دو چیست؟
- Bayes Net و RL هر دو با uncertainty سروکار دارند. uncertainty در هر کدام از چه نوعی است؟
- اگر بخواهید بازی‌ها را سخت‌تر کنید، برای هر مرحله یک تغییر پیشنهادی بنویسید.
- کدام الگوریتم از نظر دیباگ برای شما سخت‌تر بود و چرا؟

سوال اختیاری (امتیازی)

یکی از چهار مرحله را انتخاب کنید و یک بهبود مناسب پیشنهاد دهید. مثال‌ها:

- یک heuristic جدید برای CSP
- یک reward design متفاوت برای MDP
- یک schedule بهتر برای کاهش epsilon در RL
- یک نوع sensor جدید برای Bayes Net

برای بهبود پیشنهادی، توضیح دهید چه تغییری در کدام بخش لازم است اعمال شود و انتظار دارید خروجی چگونه تغییر کند. تحلیل کنید.

در مجموع، گزارش نهایی باید تصویری روشن از فرایند طراحی، توسعه و پیاده‌سازی پروژه، تجزیه و تحلیل عملکردها و مقایسه‌های مدنظر ارائه دهد، صرفاً ذکر نتایج عددی و... کافی نیست و باید به طور کامل و جامع، تجزیه و تحلیل نمائید.

تست‌های پیشنهادی

تست CSP

- دکمه **Solve** را بزنید.
- بررسی کنید همه رله‌ها روشن شده‌اند.
- ledger باید دقیقاً quota را نشان دهد.
- conflict باید صفر باشد.
- خروجی روش‌های **LCV**، **MRV** و **Degree** را مقایسه کنید.

تست MDP

- دکمه **Step** را چند بار بزنید و ببینید value ها تغییر می کنند.
- دکمه **Solve** را بزنید و منتظر مسیر پایدار بمانید.
- دکمه **Drift** را تغییر دهید و دوباره solve کنید.
- با کلیدهای حرکت، مسیر پیشنهادی را دستی امتحان کنید.

تست RL

- دکمه **Run** را بزنید و ببینید episode زیاد می شود.
- دکمه **Train50** را چند بار بزنید.
- بین **Q-Path** و **Trail** جابه جا شوید.
- ببینید مسیر آخر و فلش ها پس از آموزش تغییر می کنند یا نه.

تست BayesNet

- چند بار **Ping** بزنید.
- چند بار **Flux** بزنید.
- به تغییر MAP و entropy توجه کنید.
- با کلیک روی یک خانه حدس بزنید.
- با **New** یک anomaly جدید بسازید و دوباره امتحان کنید.

خروجی مورد انتظار برای تحویل

تحویل نهایی باید شامل موارد زیر باشد:

۱. **chronos_csp_todo.py** تکمیل شده
۲. **chronos_mdp_todo.py** تکمیل شده
۳. **chronos_rl_todo.py** تکمیل شده
۴. **chronos_bayesnet_todo.py** تکمیل شده
۵. گزارش جامع پروژه طبق موارد خواسته شده
۶. حداقل یک تصویر و توضیح اجرای موفق از هر مرحله